하네스엔지니어링 실습

💡 프로젝트명: JobTracker

> 주제: 취업준비생의 채용 지원 현황을 관리하는 웹 애플리케이션

▼ I. 프로젝트 사전 준비

1. 목표

JobTracker MVP가 도달해야 할 상태

준비물	지금 상태
입력 데이터	있다. 회사명, 지원 직무, 지원일, 지원 상태
기능 요구사항	있다. 등록, 목록 조회, 상태 관리
화면 설계	없다. 간단하게 정하면 된다
완료 조건	있다. HTML로 등록·조회, SQLite 저장, pytest 통과

1) 입력데이터

```
1. seed_applications.csv
앱이 처음 시작할 때 들어 있을 지원 내역 12건이다. 엑셀에서도 열 수 있
는 표 형식이다.

2. seed_applications.json
1번과 내용이 똑같고 형식만 JSON이다. 프로그램이 읽기 쉬운 형식이다.
두 파일 중 하나만 쓰면 된다.

3. api_test_cases.json
"이렇게 요청하면 이렇게 응답해야 한다"를 정리한 테스트 케이스 12개다.
앱이 제대로 만들어졌는지 판단하는 채점 기준, 곧 완료 조건 역할을 한다.
```

2) 기능 정의

ID	기능	사용자 관점 설명	근거 데이터
F1	초기 데이터 적재	앱을 처음 실행하면 기존 지원 내역 12건이 들어 있다	seed 파일, TC04·TC10의 전제
F2	지원 등록	회사명, 직무, 지원일, 상태, 메모를 입력해 새 지원 건을 저장한다	TC01
F3	지원 목록 조회	저장된 지원 건 전체를 지원일 최신순으로 본다	요구사항
F4	지원 상태 변경	지원 건 하나의 진행 상태를 바꾼다	TC10~12
F5	화면	목록 화면과 등록 화면에서 F2~F4를 사용한다	요구사항 “HTML 등록·조회”

3) 데이터 규칙

규칙	내용	관련 TC
필수 입력	회사명, 직무, 지원일, 상태는 필수이고 메모는 선택이다	TC02
공백 처리	양끝 공백을 제거하며, 공백만 있으면 입력이 없는 것으로 본다	TC03
날짜	YYYY-MM-DD 형식이어야 하고 실제로 있는 날짜여야 한다	TC08, TC09
상태 값	`applied`, `document_passed`, `interview`, `final_passed`, `rejected` 중 하나	TC07, TC12
중복 금지	회사명과 직무 조합이 같으면 등록하지 않는다. 공백을 제거하고, 대소문자를 무시하고 비교한다	TC04~06
저장	SQLite에 저장되어 서버를 재시작해도 남는다	요구사항

- 이번에 만들지 않는 것
  - 범위를 명시해 두지 않으면 AI가 알아서 기능을 더 붙이는 경우가 많다.
  - 지원 건 수정과 삭제
  - 로그인과 회원 관리
  - 검색, 필터, 통계

4) 최소 UI 설계 예시

화면 1. 목록 (`/`)

- 지원 건 표: 회사명, 직무, 지원일, 상태, 메모

- 행마다 상태 변경 드롭다운과 [변경] 버튼
- 상단에 [새 지원 등록] 버튼

화면 2. 등록 (`/new`)

- 입력 칸: 회사명, 직무, 지원일(날짜 선택), 상태(드롭다운), 메모
- [저장] 버튼을 누르면 목록 화면으로 이동한다
- 오류(중복, 형식 오류)가 나면 같은 화면에 메시지를 표시한다

공통 규칙

- 순수 HTML과 Jinja2 템플릿만 쓰고, React 같은 프론트엔드 프레임워크는 쓰지 않는다
- 화면은 위 2개만 만든다

2. 사용도구(일할 Agent)

Claude CLI, Claude Desktop, Codex Desktop, Codex CLI 등

3. 사용 기술 스택

- 파이썬 : 3.12
- 가상환경 : uv

```
uv init --bare python 3.12 & pin python 3.12
```

구분	FastAPI 안
웹 프레임워크	FastAPI + Uvicorn
HTML 렌더링	Jinja2 (`Jinja2Templates`)
폼 입력	`Form(...)` + `python-multipart`
DB	동일
테스트	`fastapi.testclient.TestClient` (`httpx` 필요)

4. 일할 장소 베이스 폴더

```
harness_eng/jobtracker-harness/
```

▼ II. 환경셋팅

1. 가상환경 만들기

```
cd E:\happymaker_lab\harness_eng\jobtracker-harness

uv init --bare --python 3.12    # pyproject.toml만 생성 (r
equires-python >=3.12)
uv python pin 3.12              # .python-version 파일로
3.12 고정
uv venv                         # .venv 생성 (3.12가 없으면
자동 설치)
uv run python --version         # Python 3.12.x 확인
```

2. 폴더 구조 만들기

```
jobtracker-harness/
├─ CLAUDE.md
├─ .claude/
│  ├─ settings.json
│  └─ hooks/verify_stop.py
├─ input_data/               # 추가: 이미 있는 입력 데이터 3개
├─ app/
│  ├─ main.py               # 라우트
│  ├─ db.py                 # get_db(), 스키마 생성, 시드 적
재
│  └─ templates/            # index.html, new.html
├─ tests/                   # api_test_cases.json 기반, A
I 수정 금지
│  ├─ conftest.py           # dependency_overrides + tmp
_path DB
│  └─ test_app.py
├─ pyproject.toml           # 변경: requirements.txt 대신
(uv)
├─ uv.lock
└─ .python-version
```

3. 기본 라이브러리 설치

```
uv add fastapi uvicorn jinja2 python-multipart
uv add --dev pytest httpx
```

▼ III. 추가 사항

1. 완료 조건

- `api_test_cases.json` 12개 케이스를 옮긴 pytest가 전부 통과한다.
- 서버를 켜면 브라우저에서 목록 조회, 등록, 상태 변경이 된다.
- 서버를 재시작해도 데이터가 SQLite에 남아 있다.

2. UI 설계

앞에서 본 최소 설계 그대로 간다. 화면은 목록(`/`)과 등록(`/new`) 2개이고, 순수 HTML과 Jinja2만 쓴다.

3. 남은 결정 3가지

결정	추천	이유
목록 정렬	지원일 최신순	최근 지원 건을 가장 자주 확인한다
상태 표시	화면은 한글, 저장은 영문 코드	사용자는 읽기 쉽고, 데이터와 테스트는 그대로 쓸 수 있다
미래 날짜	허용하지 않음	지원일은 이미 지원한 날이므로 미래일 수 없다

상태의 한글 표기는 이렇게 맞춘다. `applied`는 지원완료, `document_passed`는 서류합격, `interview`는 면접, `final_passed`는 최종합격, `rejected`는 불합격이다.

4. 하지 않을 것

- 지원 건 수정과 삭제
- 로그인과 회원 관리
- 검색, 필터, 통계
- React 같은 프론트엔드 프레임워크

미래 날짜 금지는 `api_test_cases.json`에 없는 규칙이다. 이 규칙을 넣으려면 테스트 케이스를 하나 추가해야 한다. 추가할 테스트를 늘리고 싶지 않다면 "허용"으로 정해도 문제없다.